SystemVerilog and UVM Practice Journey

Phase 01 — SystemVerilog Fundamentals

Bibhu Asish Panda

November 4, 2026

Contents

1 Phase 01 — SystemVerilog Fundamentals 1
1.1 Overview . . . 1
1.2 Topics Covered . . . 1
1.2.1 Data Types and Type Operations . . . 1
1.3 Key Learning Areas . . . 1
1.4 Practical Work . . . 2
1.5 Learning Outcomes . . . 2
1.6 Tools . . . 2
1.7 SV-001 — Data Types . . . 3
1.7.1 Concept . . . 3
1.7.2 Important Difference . . . 4
1.7.3 Verification Relevance . . . 4
1.7.4 Complete Example . . . 4
1.7.5 Expected Output . . . 5
1.7.6 Key Points to Remember . . . 5
1.8 SV-002 — 2-State vs 4-State . . . 6
1.8.1 Concept . . . 6
1.8.2 Why X/Z Matter . . . 6
1.8.3 Complete Example . . . 7
1.8.4 Expected Output . . . 7
1.8.5 Verification Relevance . . . 8
1.8.6 Key Points to Remember . . . 8
1.9 SV-003 — Signed/Unsigned . . . 9
1.9.1 Concept . . . 9
1.9.2 Sign Extension . . . 9
1.9.3 Arithmetic . . . 10
1.9.4 Syntax . . . 10
1.9.5 Complete Example . . . 10
1.9.6 Expected Output . . . 11
1.9.7 Verification Relevance . . . 11
1.9.8 Key Points to Remember . . . 11
1.10 SV-004 — Strings . . . 12
1.10.1 Concept . . . 12
1.10.2 Declaration and Assignment . . . 12
1.10.3 String Methods . . . 12
1.10.4 String Comparison . . . 12
1.10.5 String Formatting . . . 13

1.10.6 Complete Example . . . 13
1.10.7 Expected Output . . . 13
1.10.8 Verification Relevance . . . 14
1.10.9 Key Points to Remember . . . 14
1.11 SV-005 — Parameters . . . 15
1.11.1 Concept . . . 15
1.11.2 `parameter` . . . 15
1.11.3 `localparam` . . . 15
1.11.4 Parameterized Values . . . 15
1.11.5 `parameter` vs `localparam` . . . 16
1.11.6 Complete Example . . . 16
1.11.7 Expected Output . . . 16
1.11.8 Verification Relevance . . . 17
1.11.9 Key Points to Remember . . . 17
1.12 SV-006 — Typedef . . . 18
1.12.1 Concept . . . 18
1.12.2 Basic `typedef` . . . 18
1.12.3 User-Defined Types . . . 18
1.12.4 `typedef enum` . . . 18
1.12.5 Enum `.name()` . . . 19
1.12.6 Complete Example . . . 19
1.12.7 Expected Output . . . 20
1.12.8 Verification Relevance . . . 20
1.12.9 Key Points to Remember . . . 20
1.13 SV-007 — Enum . . . 21
1.13.1 Concept . . . 21
1.13.2 Enum Declaration . . . 21
1.13.3 Enum Methods . . . 21
1.13.4 Complete Example . . . 23
1.13.5 Expected Output . . . 23
1.13.6 Verification Relevance . . . 23
1.13.7 Key Points to Remember . . . 24
1.14 SV-008 — Struct . . . 25
1.14.1 Struct . . . 25
1.14.2 Complete Runnable Code . . . 26
1.14.3 Expected Output . . . 26
1.14.4 Key Syntax . . . 27
1.15 SV-009 — Union . . . 28
1.15.1 Union . . . 28
1.15.2 Complete Runnable Code . . . 29
1.15.3 Expected Output . . . 30
1.16 SV-010 — Type Casting . . . 31
1.16.1 Type Casting . . . 31
1.16.2 Complete Runnable Code . . . 32
1.16.3 Expected Output . . . 32
1.16.4 Key Syntax . . . 33

1.17 SV-011 — `$cast` . . . 34
1.17.1 Dynamic Casting with `$cast` . . . 34
1.17.2 Complete Runnable Code . . . 35
1.17.3 Expected Output . . . 36
1.17.4 Key Syntax . . . 37
1.18 SV-012 — System Functions . . . 38
1.18.1 `$bits` . . . 38
1.18.2 `$size` . . . 38
1.18.3 `$clog2` . . . 39
1.18.4 `$left` and `$right` . . . 39
1.18.5 `$high` and `$low` . . . 39
1.18.6 `$left` / `$right` vs `$high` / `$low` . . . 40
1.18.7 Why These Functions Matter in Verification . . . 40
1.18.8 Complete Runnable Code . . . 40
1.18.9 Expected Output . . . 41
1.18.10 Key Syntax . . . 41

Chapter 1

Phase 01 — SystemVerilog Fundamentals

1.1 Overview

This phase introduces the fundamental building blocks of SystemVerilog, focusing on data types, type definitions, type conversion, and built-in system functions. These concepts form the foundation for writing readable, reusable, and reliable RTL and verification code.

1.2 Topics Covered

1.2.1 Data Types and Type Operations

No.	Topic	Concepts Covered
SV-001	Data Types	`bit`, `logic`, `reg`, `byte`, `shortint`, `int`, `longint`, `integer`, `time`
SV-002	2-State vs 4-State	`bit` vs `logic`, X/Z values and their behavior
SV-003	Signed/Unsigned	`signed`, `unsigned`, sign extension, arithmetic behavior
SV-004	Strings	Declaration, assignment, string methods, comparison, formatting
SV-005	Parameters	`parameter`, `localparam`, parameterized values
SV-006	Typedef	User-defined types, `typedef`, `typedef enum`
SV-007	Enum	`enum`, `first`, `last`, `next`, `prev`, `name`, `num`
SV-008	Struct	`struct`, packed structures, unpacked structures
SV-009	Union	`union`, packed unions, shared storage
SV-010	Type Casting	Static casting, `int'`, `logic'`, type conversion
SV-011	`$cast`	Dynamic casting, safe casting, class handles
SV-012	System Functions	`$bits`, `$size`, `$clog2`, `$left`, `$right`, `$high`, `$low`

1.3 Key Learning Areas

- **Data representation:** Understand SystemVerilog data types and the differences between 2-state and 4-state values.

- **Type behavior:** Learn signed and unsigned arithmetic, sign extension, and type conversion.
- **Custom types:** Create reusable definitions using `typedef`, `enum`, `struct`, and `union`.
- **String handling:** Perform common string operations and formatting.
- **Type safety:** Understand static casting and dynamic casting with `$cast`.
- **System functions:** Use built-in functions to inspect data widths, array dimensions, and ranges.

1.4 Practical Work

Each topic is practiced through SystemVerilog code examples, with simulation in Vivado to observe language behavior and verify expected outputs.

1.5 Learning Outcomes

After completing this phase, you should be able to:

- Select appropriate SystemVerilog data types.
- Explain X/Z behavior and signed/unsigned arithmetic.
- Define and use custom data types.
- Apply type casting and common system functions.
- Write and simulate basic SystemVerilog programs.

1.6 Tools

- **Language:** SystemVerilog
- **Simulator:** AMD Vivado
- **Practice:** Runnable code examples, expected outputs, and GitHub learning notes

1.7 SV-001 — Data Types

1.7.1 Concept

SystemVerilog provides several built-in data types for representing logic values, integers, and simulation time.

1. `bit` `bit` is a **2-state** data type. It can contain only:

```
0
1
```

It cannot retain `X` or `Z`.

```
bit b;
```

2. `logic` `logic` is a **4-state** data type. It can represent:

```
0, 1, X, Z
```

It is commonly used in SystemVerilog instead of the older `reg` type.

```
logic signal;
```

3. `reg` `reg` is the traditional Verilog variable type. It is a **4-state** type:

```
0, 1, X, Z
```

SystemVerilog generally uses `logic` instead of `reg`, but `reg` is still important when reading or understanding older Verilog code.

Integer Data Types

Type	Width	State	Default signedness
`byte`	8-bit	2-state	signed
`shortint`	16-bit	2-state	signed
`int`	32-bit	2-state	signed
`longint`	64-bit	2-state	signed
`integer`	32-bit	4-state	signed
`time`	64-bit	4-state	unsigned

`int` is particularly common in verification code for counters, loop variables, calculations, and temporary values.

`time` `time` is used for storing simulation-time values.

Example:

```
#10;
t = $time;
```

The value of `$time` is assigned to the `time` variable.

1.7.2 Important Difference

The main distinction to remember is:

```
2-state -> 0, 1
4-state -> 0, 1, X, Z
```

Therefore:

```
bit   -> 2-state
logic -> 4-state
reg   -> 4-state
```

The integer atom types `byte`, `shortint`, `int`, and `longint` are 2-state types, while `integer` and `time` are 4-state types.

1.7.3 Verification Relevance

These data types are used throughout SystemVerilog verification:

- `logic` for DUT/interface signals
- `bit` when X/Z states are not required
- `int` for counters and verification variables
- `byte` for byte-oriented data
- `longint` for larger numerical values
- `time` for simulation-time measurements
- Integer types inside transactions, constraints, monitors, scoreboards, coverage, and UVM components

1.7.4 Complete Example

```
module sv_001_data_types;

    bit       b;       // 2-state: 0, 1
    logic     l;       // 4-state: 0, 1, X, Z
    reg       r;       // 4-state, legacy Verilog type

    byte      by;      // 8-bit signed integer
    shortint  si;      // 16-bit signed integer
    int       i;       // 32-bit signed integer
    longint   li;      // 64-bit signed integer
    integer   iv;      // 32-bit signed, 4-state
    time      t;       // 64-bit unsigned simulation time

    initial begin
        b  = 1'b1;
        l  = 1'bx;
        r  = 1'bz;

        by = 8'd100;
        si = 16'd1000;
        i  = 32'd100000;
```

```
        li = 64'd1000000;
        iv = -50;
        #10;
        t = $time;

        $display("bit       = %b", b);
        $display("logic     = %b", l);
        $display("reg       = %b", r);
        $display("byte      = %0d", by);
        $display("shortint  = %0d", si);
        $display("int       = %0d", i);
        $display("longint   = %0d", li);
        $display("integer   = %0d", iv);
        $display("time      = %0t", t);
    end
endmodule
```

1.7.5 Expected Output

```
bit       = 1
logic     = x
reg       = z
byte      = 100
shortint  = 1000
int       = 100000
longint   = 1000000
integer   = -50
time      = 10
```

1.7.6 Key Points to Remember

- `bit` → 2-state.
- `logic` → 4-state.
- `reg` → legacy 4-state Verilog type.
- `byte` → 8-bit.
- `shortint` → 16-bit.
- `int` → 32-bit.
- `longint` → 64-bit.
- `integer` → 32-bit, 4-state signed type.
- `time` → used for simulation time.
- `logic` is generally preferred over `reg` in SystemVerilog.
- Choosing the correct data type is important for reliable verification code.

1.8 SV-002 — 2-State vs 4-State

1.8.1 Concept

SystemVerilog data types can be broadly classified as **2-state** or **4-state** types.

2-State Types A 2-state type can represent only:

```
0
1
```

Example:

```
bit signal;
```

`bit` does not retain `X` or `Z`.

4-State Types A 4-state type can represent:

```
0
1
X
Z
```

Example:

```
logic signal;
```

Here:

- `0` → logic low
- `1` → logic high
- `X` → unknown/uninitialized/conflicting value
- `Z` → high-impedance state

`bit` vs `logic`

Type	States	Typical Use
`bit`	`0`, `1`	2-state verification variables
`logic`	`0`, `1`, `X`, `Z`	RTL and verification signals

When an `X` or `Z` value is assigned to a `bit`, it cannot retain that state. The value is converted to a valid 2-state value. A `logic` variable preserves the `X` or `Z` value.

1.8.2 Why X/Z Matter

`X` can indicate situations such as:

- An uninitialized signal
- Multiple conflicting drivers
- Unknown hardware state
- A reset or initialization problem

`z` commonly represents:

- High-impedance
- A disconnected/tri-stated signal
- Tri-state bus behavior

Therefore, 4-state simulation can expose problems that may be hidden when using only 2-state variables.

1.8.3 Complete Example

```
module sv_002_2state_vs_4state;

    bit   bit_signal;       // 2-state: only 0, 1
    logic logic_signal;     // 4-state: 0, 1, X, Z

    initial begin
        // X/Z cannot be retained by a 2-state variable
        bit_signal   = 1'bx;
        logic_signal = 1'bx;

        $display("After assigning X:");
        $display("bit   = %b", bit_signal);
        $display("logic = %b", logic_signal);

        bit_signal   = 1'bz;
        logic_signal = 1'bz;

        $display("\nAfter assigning Z:");
        $display("bit   = %b", bit_signal);
        $display("logic = %b", logic_signal);

        bit_signal   = 1'b1;
        logic_signal = 1'b1;

        $display("\nAfter assigning 1:");
        $display("bit   = %b", bit_signal);
        $display("logic = %b", logic_signal);
    end
endmodule
```

1.8.4 Expected Output

```
After assigning X:
bit   = 0
logic = x

After assigning Z:
bit   = 0
logic = z
```

```
8
9  After assigning 1:
10 bit   = 1
11 logic = 1
```

1.8.5 Verification Relevance

Understanding 2-state vs 4-state behavior is important when writing testbenches, monitors, scoreboards, assertions, and UVM components.

Using `logic` where X/Z behavior matters allows the verification environment to detect unknown or high-impedance conditions instead of silently losing that information.

1.8.6 Key Points to Remember

- `bit` → 2-state: `0/1`.
- `logic` → 4-state: `0/1/X/Z`.
- `X` represents an unknown state.
- `Z` represents high impedance.
- `bit` cannot retain `X` or `Z`.
- `logic` preserves `X` and `Z`.
- 4-state simulation is useful for finding initialization and connectivity problems.

1.9 SV-003 — Signed/Unsigned

1.9.1 Concept

SystemVerilog values can be declared as either **signed** or **unsigned**.

This affects how the same binary bits are interpreted during:

- Arithmetic
- Comparisons
- Assignments
- Size conversion
- Sign extension

Signed A signed value uses its **most significant bit (MSB)** as the sign bit.

```
logic signed [7:0] value;
```

For an 8-bit signed value:

```
00000101 -> +5
11111011 -> -5
```

Unsigned An unsigned value treats all bits as magnitude.

```
logic [7:0] value;
```

Therefore:

```
11111011 -> 251
```

The same 8-bit pattern can therefore represent different numbers:

```
Signed   -> -5
Unsigned -> 251
```

1.9.2 Sign Extension

When a signed value is expanded to a larger width, the sign bit is copied into the newly added upper bits.

Example:

```
8-bit  -5:
11111011

16-bit after sign extension:
1111111111111011
```

The value remains `-5`. For a positive signed value, the upper bits are filled with `0`.

```
8-bit  +5:
00000101

16-bit:
0000000000000101
```

This preserves the original signed value.

1.9.3 Arithmetic

Signedness affects arithmetic interpretation.

For example:

```
8'b11111011
```

is:

```
-5    -> signed
251   -> unsigned
```

Therefore arithmetic performed using these values can produce different numerical results.

1.9.4 Syntax

Signed declaration:

```
logic signed [7:0] a;
```

Unsigned declaration:

```
logic unsigned [7:0] b;
```

`unsigned` is also the default for `logic` unless `signed` is specified.

1.9.5 Complete Example

```
module sv_003_signed_unsigned;

    logic signed [7:0]   signed_8;
    logic        [7:0]   unsigned_8;

    logic signed [15:0]  signed_16;
    logic        [15:0]  unsigned_16;

    initial begin
        signed_8   = -8'sd5;
        unsigned_8 = 8'd251;       // Same bits as -5

        // Sign extension: signed value extends with its sign bit
        signed_16   = signed_8;
        unsigned_16 = unsigned_8;

        $display("signed_8    = %0d", signed_8);
        $display("unsigned_8  = %0d", unsigned_8);

        $display("signed_16   = %0d", signed_16);
        $display("unsigned_16 = %0d", unsigned_16);

        $display("signed 8 + 3   = %0d", signed_8 + 8'sd3);
        $display("unsigned 8 + 3 = %0d", unsigned_8 + 8'd3);
    end

endmodule
```

1.9.6 Expected Output

```
signed_8    = -5
unsigned_8  = 251
signed_16   = -5
unsigned_16 = 251
signed 8 + 3   = -2
unsigned 8 + 3 = 254
```

1.9.7 Verification Relevance

Signedness is important in verification because transaction fields, counters, arithmetic calculations, comparisons, and reference models may contain both positive and negative values.

Incorrect signedness can cause:

- Unexpected arithmetic results
- Incorrect comparisons
- Incorrect sign extension
- Scoreboard mismatches
- Incorrect reference-model calculations

1.9.8 Key Points to Remember

- `signed` interprets the MSB as a sign bit.
- `unsigned` treats all bits as magnitude.
- `logic` is unsigned by default unless declared `signed`.
- The same binary pattern can represent different signed and unsigned values.
- Sign extension copies the signed value's MSB into the new bits.
- Zero extension fills added upper bits with `0`.
- Always check signedness and width when performing arithmetic.

1.10 SV-004 — Strings

1.10.1 Concept

SystemVerilog provides the built-in `string` type for storing and manipulating text.

A string can contain a sequence of characters and can be modified using built-in string methods.

1.10.2 Declaration and Assignment

A string is declared using the `string` keyword:

```
string name;
string message;
```

Values can then be assigned using double quotes:

```
name = "Bibhu";
```

Unlike fixed-width character vectors, SystemVerilog strings are dynamic in length.

1.10.3 String Methods

Some commonly used string methods are:

`len()` Returns the number of characters in the string.

```
message.len()
```

`toupper()` Returns a version of the string converted to uppercase.

```
message.toupper()
```

`tolower()` Returns a version of the string converted to lowercase.

```
message.tolower()
```

These methods are useful for processing and checking text in verification environments.

1.10.4 String Comparison

Strings can be compared using operators such as `==` and `!=`.

Example:

```
if (name == "Bibhu")
    $display("MATCH");
```

This is useful for checking names, commands, configuration values, test names, and other text-based information.

1.10.5 String Formatting

Strings can be displayed using `$display()` and the `%s` format specifier.

```
$display("Name: %s", name);
```

Multiple values can also be combined:

```
$display("Name: %s | Topic: %s", name, message);
```

Formatting is especially useful for readable simulation logs and debug messages.

1.10.6 Complete Example

```
module sv_004_strings;

    string name;
    string message;

    initial begin
        // Declaration and assignment
        name    = "Bibhu";
        message = "SystemVerilog";

        $display("name    = %s", name);
        $display("message = %s", message);

        // String methods
        $display("length  = %0d", message.len());

        message = message.toupper();
        $display("upper   = %s", message);

        message = message.tolower();
        $display("lower   = %s", message);

        // Comparison
        if (name == "Bibhu")
            $display("comparison = MATCH");
        else
            $display("comparison = NO MATCH");

        // Formatting
        $display("formatted = Name: %s | Topic: %s", name, message);
    end
endmodule
```

1.10.7 Expected Output

```
name    = Bibhu
message = SystemVerilog
length  = 13
```

```
upper   = SYSTEMVERILOG
lower   = systemverilog
comparison = MATCH
formatted = Name: Bibhu | Topic: systemverilog
```

1.10.8 Verification Relevance

Strings are commonly used in verification for:

- Test and transaction names
- Debug messages
- Configuration values
- File and path information
- Protocol or command names
- Simulation log formatting
- UVM reporting and messages

They are particularly useful when verification code needs to work with human-readable information.

1.10.9 Key Points to Remember

- Declare strings using `string`.
- Assign text using double quotes.
- `len()` returns the string length.
- `toupper()` converts text to uppercase.
- `tolower()` converts text to lowercase.
- `==` and `!=` can be used for string comparison.
- `%s` is used to format strings with `$display()`.
- Strings are primarily useful in testbench and verification code.

1.11 SV-005 — Parameters

1.11.1 Concept

A `parameter` is a constant value that can be used to configure a module or other SystemVerilog construct. Parameters are useful when the same design needs to work with different sizes or configurations.

1.11.2 `parameter`

A parameter can be declared inside a module:

```
parameter int WIDTH = 8;
```

The default value is `8`, but it can be overridden when the module is instantiated.

Example:

```
sv_005_parameters #(
    .WIDTH(16)
) dut ();
```

The module now uses `WIDTH = 16`.

1.11.3 `localparam`

A `localparam` is a constant value that is local to the module and **cannot be overridden during module instantiation**.

Example:

```
localparam int ADDR_WIDTH = $clog2(DEPTH);
```

This is useful when a value must be calculated or defined internally from other parameters.

For example, if:

```
DEPTH = 64
```

then:

```
ADDR_WIDTH = $clog2(64) = 6
```

1.11.4 Parameterized Values

Parameters can control widths, depths, sizes, and other design configuration values.

Example:

```
logic [WIDTH-1:0] data;
```

If:

```
WIDTH = 8
```

then `data` is 8 bits wide.

If:

```
WIDTH = 16
```

then `data` becomes 16 bits wide.

This allows one module to be reused with different configurations.

1.11.5 `parameter` vs `localparam`

Feature	`parameter`	`localparam`
Constant value	Yes	Yes
Default value	Yes	Yes
Can be overridden	Yes	No
Common use	Module configuration	Internal derived constants

1.11.6 Complete Example

```
module sv_005_parameters #(
    parameter int WIDTH = 8,       // Can be overridden
    parameter int DEPTH = 16
);

    localparam int ADDR_WIDTH = $clog2(DEPTH); // Cannot be overridden
    logic [WIDTH-1:0] data;
    logic [ADDR_WIDTH-1:0] address;

    initial begin
        data    = '1;
        address = '0;

        $display("WIDTH      = %0d", WIDTH);
        $display("DEPTH      = %0d", DEPTH);
        $display("ADDR_WIDTH = %0d", ADDR_WIDTH);
        $display("data width = %0d bits", $bits(data));
        $display("addr width = %0d bits", $bits(address));
    end
endmodule


module tb;

    // Override the parameterized values
    sv_005_parameters #(
        .WIDTH(16),
        .DEPTH(64)
    ) dut ();
endmodule
```

1.11.7 Expected Output

```
WIDTH      = 16
DEPTH      = 64
ADDR_WIDTH = 6
data width = 16 bits
addr width = 6 bits
```

1.11.8 Verification Relevance

Parameters are useful in verification for creating reusable testbench components and configuring designs with different:

- Data widths
- FIFO depths
- Address widths
- Number of transactions
- Buffer sizes
- Protocol configurations

Parameterized designs reduce duplicated code and make the same verification environment easier to reuse with different configurations.

1.11.9 Key Points to Remember

- `parameter` defines a configurable constant.
- Parameters have default values.
- Module parameters can be overridden during instantiation.
- `localparam` defines a constant that cannot be overridden.
- Parameters are commonly used for widths, depths, and configurations.
- `localparam` is useful for internally derived values.
- `$clog2()` can be used to calculate address widths from depths.

1.12 SV-006 — Typedef

1.12.1 Concept

`typedef` is used to create a **user-defined name for an existing data type**.

Instead of repeatedly writing a complex type, we can define it once and use a shorter, meaningful name.

1.12.2 Basic `typedef`

Example:

```
typedef logic [7:0] data_t;
```

Now `data_t` represents:

```
logic [7:0]
```

Variables can be declared using the new type:

```
data_t data;
```

This improves readability and makes the type easier to reuse.

1.12.3 User-Defined Types

Multiple types can be defined using `typedef`.

```
typedef logic [7:0] data_t;
typedef int unsigned count_t;
```

These names can then be used throughout the code:

```
data_t  data;
count_t count;
```

The underlying data types remain `logic [7:0]` and `int unsigned`, but the new names describe their purpose more clearly.

1.12.4 `typedef enum`

`typedef enum` creates a named enumeration type.

Example:

```
typedef enum {
    IDLE,
    START,
    DONE
} state_t;
```

Now `state_t` is a user-defined enum type.

A variable can be declared:

```
state_t state;
```

and assigned one of its named values:

```
state = IDLE;
state = START;
state = DONE;
```

Enums are much more readable than representing states with raw no.s such as `0`, `1`, and `2`.

1.12.5 Enum `.name()`

An enum variable can use the `.name()` method to return its enumeration name as a string.

Example:

```
$display("%s", state.name());
```

For `state = START`, the output is:

```
START
```

1.12.6 Complete Example

```
module sv_006_typedef;
    // Create user-defined type names
    typedef logic [7:0] data_t;
    typedef int unsigned count_t;

    // Create an enum type for named states
    typedef enum {
        IDLE,
        START,
        DONE
    } state_t;

    data_t  data;
    count_t count;
    state_t state;

    initial begin
        data  = 8'hA5;
        count = 100;
        state = IDLE;

        $display("data  = %h", data);
        $display("count = %0d", count);
        $display("state = %s", state.name());

        state = START;
        $display("state = %s", state.name());

        state = DONE;
        $display("state = %s", state.name());
    end
endmodule
```

1.12.7 Expected Output

```
data  = a5
count = 100
state = IDLE
state = START
state = DONE
```

1.12.8 Verification Relevance

`typedef` is heavily used in SystemVerilog verification to create meaningful and reusable types for:

- Transaction fields
- Protocol data
- Counters
- Configuration values
- FSM/protocol states
- Testbench states
- UVM sequence items and related types

`typedef enum` is particularly useful when verification code needs to represent a finite set of meaningful states.

1.12.9 Key Points to Remember

- `typedef` creates a new name for an existing type.
- User-defined types improve readability and reuse.
- The `_t` suffix is commonly used for type names.
- `typedef enum` creates a named enumeration type.
- Enum values provide readable symbolic names.
- `.name()` returns the current enum value as a string.
- Enums are useful for FSM and protocol state representation.

1.13 SV-007 — Enum

1.13.1 Concept

An `enum` (enumeration) defines a set of named values.

Instead of using unexplained numeric values:

```
0 -> IDLE
1 -> START
2 -> ACTIVE
3 -> DONE
```

we can use meaningful names directly:

```
state = ACTIVE;
```

This makes RTL and verification code easier to understand and debug.

1.13.2 Enum Declaration

An enum can be declared using:

```
typedef enum {
    IDLE,
    START,
    ACTIVE,
    DONE
} state_t;
```

Then a variable of this enum type can be declared:

```
state_t state;
```

By default, enum values are assigned increasing values starting from `0`.

Therefore:

```
IDLE   -> 0
START  -> 1
ACTIVE -> 2
DONE   -> 3
```

1.13.3 Enum Methods

SystemVerilog provides built-in methods for examining and navigating enum values.

`first()` Returns the **first enum value**.

```
state.first()
```

For the example:

```
IDLE
```

`last()` Returns the **last enum value**.

```
state.last()
```

For the example:

```
DONE
```

`next()` Returns the **next enum value** relative to the current value.
If:

```
state = ACTIVE
```

then:

```
state.next() -> DONE
```

`prev()` Returns the **previous enum value** relative to the current value.
If:

```
state = ACTIVE
```

then:

```
state.prev() -> START
```

`name()` Returns the name of the current enum value as a string.

```
state.name()
```

If:

```
state = ACTIVE
```

then:

```
state.name() -> "ACTIVE"
```

This is particularly useful for readable debug messages.

`num()` Returns the total number of values in the enum.
For:

```
IDLE, START, ACTIVE, DONE
```

the result is:

```
4
```

1.13.4 Complete Example

```
module sv_007_enum;

    typedef enum {
        IDLE,
        START,
        ACTIVE,
        DONE
    } state_t;

    state_t state;

    initial begin
        state = ACTIVE;

        $display("Current = %s", state.name());
        $display("First   = %s", state.first().name());
        $display("Last    = %s", state.last().name());
        $display("Next    = %s", state.next().name());
        $display("Previous= %s", state.prev().name());
        $display("Number  = %0d", state.num());
    end

endmodule
```

1.13.5 Expected Output

```
Current = ACTIVE
First   = IDLE
Last    = DONE
Next    = DONE
Previous= START
Number  = 4
```

1.13.6 Verification Relevance

Enums are useful for representing a fixed set of meaningful states, such as:

- FSM states
- Protocol states
- Transaction states
- Testbench modes
- Driver/monitor states
- UVM-related state information

The enum methods are especially useful for writing readable debug messages and navigating through defined states.

1.13.7 Key Points to Remember

- `enum` defines a fixed set of named values.
- Enum values are assigned in declaration order by default.
- `first()` returns the first enum value.
- `last()` returns the last enum value.
- `next()` returns the next value.
- `prev()` returns the previous value.
- `name()` returns the current enum value as a string.
- `num()` returns the total number of enum values.
- Enums make FSM and verification code more readable than raw numbers.

1.14 SV-008 — Struct

1.14.1 Struct

A `struct` groups multiple related variables into one data object. Instead of keeping several independent variables, related information can be represented as members of a single structure.

SystemVerilog supports both **unpacked structs** and **packed structs**.

1. Unpacked Struct An unpacked struct stores its members as separate objects. The members can have different data types and sizes.

```
typedef struct {
    string name;
    int    age;
} person_t;
```

Access members using the dot operator:

```
person.name = "Bibhu";
person.age  = 21;
```

2. Packed Struct A packed struct combines all members into one contiguous packed bit representation.

```
typedef struct packed {
    logic [3:0] opcode;
    logic [7:0] data;
} packet_t;
```

Here:

- `opcode` = 4 bits
- `data` = 8 bits
- Total packed width = 12 bits

Therefore, the complete struct can be displayed or accessed like a vector:

```
packet = 12'hA5A;
packet[11:8]
packet[7:0]
```

Why Structs Matter in Verification Structs are useful for grouping related information such as:

- Transaction fields
- Packet information
- Configuration values
- Control/status information
- Protocol data

Packed structs are especially useful when the grouped fields need to represent a hardware bit-level format.

1.14.2 Complete Runnable Code

Code type: SystemVerilog simulation/demo — no DUT.

```
module sv_008_struct;

    // Unpacked struct
    typedef struct {
        string name;
        int    age;
    } person_t;

    // Packed struct
    typedef struct packed {
        logic [3:0] opcode;
        logic [7:0] data;
    } packet_t;

    person_t person;
    packet_t packet;

    initial begin
        // Unpacked struct members
        person.name = "Bibhu";
        person.age  = 21;

        $display("Unpacked Struct:");
        $display("name = %s", person.name);
        $display("age  = %0d", person.age);

        // Packed struct members
        packet.opcode = 4'hA;
        packet.data   = 8'h5A;

        $display("\nPacked Struct:");
        $display("opcode = %h", packet.opcode);
        $display("data   = %h", packet.data);
        $display("packet = %h", packet);

        // Packed struct can be treated as one bit vector
        $display("packet[11:8] = %h", packet[11:8]);
        $display("packet[7:0]  = %h", packet[7:0]);
    end

endmodule
```

1.14.3 Expected Output

```
Unpacked Struct:
name = Bibhu
age  = 21

```

```
Packed Struct:
opcode = a
data   = 5a
packet = a5a
packet[11:8] = a
packet[7:0]  = 5a
```

1.14.4 Key Syntax

```
typedef struct {
    type member1;
    type member2;
} struct_name;
```

Unpacked struct:

```
typedef struct {
    string name;
    int age;
} person_t;
```

Packed struct:

```
typedef struct packed {
    logic [3:0] opcode;
    logic [7:0] data;
} packet_t;
```

Member access:

```
person.name
person.age

packet.opcode
packet.data
```

The main difference is that a **packed struct has one contiguous packed bit representation**, while an **unpacked struct does not form one packed vector**.

Structs become particularly useful later in SystemVerilog verification when building transaction and packet-like data objects.

1.15 SV-009 — Union

1.15.1 Union

A `union` allows multiple members to **share the same storage location**. The important difference from a `struct` is:

- **Struct:** each member has separate storage.
- **Union:** all members share the same storage.

Therefore, writing to one union member changes the value that is seen through the other members.

SystemVerilog supports both **unpacked unions** and **packed unions**.

1. Unpacked Union An unpacked union can contain members with different data types.

```
typedef union {
    int  int_value;
    byte byte_value;
} data_u;
```

Both `int_value` and `byte_value` use the same storage area.

```
data.int_value = 32'h12345678;
data.byte_value = 8'hA5;
```

When `byte_value` is written, the shared storage is updated.

2. Packed Union A packed union allows the shared storage to be treated as packed bits through different member declarations.

```
typedef union packed {
    logic [7:0] byte_value;
    logic [3:0] high_nibble;
    logic [7:0] bits;
} packed_u;
```

For example:

```
packet.bits = 8'hA5;
```

The same packed storage can then be viewed through another member:

```
packet.byte_value
packet.high_nibble
```

For `8'hA5`:

- `byte_value` = `A5`
- `high_nibble` = `5`

Struct vs Union

Feature	Struct	Union
Member storage	Separate	Shared
Members coexist independently	Yes	No
Packed version	Yes	Yes
Main purpose	Group related fields	Interpret the same storage differently

Why Union Matters in Verification Unions can be useful when the same data needs to be interpreted in different forms, such as:

- Different views of packet data
- Bit-level and field-level interpretation
- Data representation conversions
- Protocol-related data handling

The key idea is **shared storage**.

1.15.2 Complete Runnable Code

Code type: SystemVerilog simulation/demo — no DUT.

```
module sv_009_union;

    // Unpacked union
    typedef union {
        int  int_value;
        byte byte_value;
    } data_u;

    // Packed union
    // All members must have the same size: 8 bits
    typedef union packed {
        logic [7:0] byte_value;
        logic [7:0] bits;
        logic signed [7:0] signed_value;
    } packed_u;

    data_u   data;
    packed_u packet;

    initial begin
        // --------------------------------
        // Unpacked Union
        // --------------------------------
        data.int_value = 32'h12345678;

        $display("Unpacked Union:");
        $display("int_value  = %h", data.int_value);
```

```
28
29          // Same storage is now written through byte_value
30          data.byte_value = 8'hA5;
31
32          $display("byte_value = %h", data.byte_value);
33
34          // -------------------------------
35          // Packed Union
36          // -------------------------------
37          packet.bits = 8'hA5;
38
39          $display("\nPacked Union:");
40          $display("bits         = %h", packet.bits);
41          $display("byte_value   = %h", packet.byte_value);
42          $display("signed_value = %0d", packet.signed_value);
43      end
44
45  endmodule
```

1.15.3 Expected Output

```
Unpacked Union:
int_value  = 12345678
byte_value = a5

Packed Union:
bits        = a5
byte_value  = a5
high_nibble = 5
```

Key Syntax Unpacked union:

```
typedef union {
    type member1;
    type member2;
} union_name;
```

Packed union:

```
typedef union packed {
    logic [7:0] byte_value;
    logic [3:0] high_nibble;
} packed_u;
```

The central concept is:

A union provides multiple views of the same storage.

Unlike a struct, changing one union member changes the shared storage seen through the other members.

1.16 SV-010 — Type Casting

1.16.1 Type Casting

Type casting is the process of converting a value from one SystemVerilog data type to another.

SystemVerilog supports **static casting** using the target type followed by an apostrophe:

```
int'(expression)
logic'(expression)
```

The conversion happens at the point where the expression is evaluated.

Static Casting The general syntax is:

```
target_type'(expression)
```

For example:

```
int_value = int'(data);
```

This explicitly converts `data` to an `int` before assigning it.

Another example:

```
logic_value = logic'(1'b1);
```

Here, the expression is explicitly converted to the `logic` type.

`int'` `int'` converts an expression to the SystemVerilog `int` type.

```
int_value = int'(data);
```

For example, an 8-bit value:

```
8'hA5
```

has the unsigned decimal value:

```
165
```

when converted to `int`.

`logic'` `logic'` explicitly converts an expression to the `logic` type.

```
logic_value = logic'(1'b1);
```

The result is a 4-state `logic` value.

Type Conversion and Width Casting can also be used when converting between expressions of different widths.

```
extended_data = int'(data);
```

The 8-bit value is converted to `int` and then assigned to the 16-bit destination.

For:

```
data = 8'hA5
```

the resulting 16-bit value is:

```
00A5
```

Why Type Casting Matters in Verification Explicit casting is useful when:

- Converting transaction fields between types
- Controlling signed/unsigned interpretation
- Handling different signal widths
- Passing values to functions/tasks with specific types
- Avoiding unintended implicit conversions
- Making testbench code easier to understand

The important syntax to remember is:

```
target_type'(expression)
```

1.16.2 Complete Runnable Code

Code type: SystemVerilog simulation/demo — no DUT.

```
module sv_010_type_casting;

    logic [7:0]  data;
    logic [15:0] extended_data;
    int          int_value;
    logic        logic_value;

    initial begin
        data = 8'hA5;

        // Static cast to int
        int_value = int'(data);

        // Static cast to logic
        logic_value = logic'(1'b1);

        // Type conversion with a wider destination
        extended_data = int'(data);

        $display("Original data    = %h", data);
        $display("int'(data)       = %0d", int_value);
        $display("logic'(1'b1)     = %b", logic_value);
        $display("Extended data    = %h", extended_data);
    end

endmodule
```

1.16.3 Expected Output

```
Original data    = a5
int'(data)       = 165
logic'(1'b1)     = 1
Extended data    = 00a5
```

1.16.4 Key Syntax

```
int'(expression)
logic'(expression)
```

General form:

```
target_type'(expression)
```

For example:

```
int_value = int'(data);
logic_value = logic'(1'b1);
```

Static casting gives explicit control over how a value is interpreted or converted instead of relying only on implicit type conversion.

1.17 SV-011 — `$cast`

1.17.1 Dynamic Casting with `$cast`

`$cast` performs a **dynamic cast** at runtime. It is commonly used with class handles when a base-class handle may refer to either a base-class object or a derived-class object. The syntax is:

```
$cast(destination, source)
```

`$cast` returns:

- `1` when the cast succeeds
- `0` when the cast fails

This makes it a **safe casting mechanism**.

Class Handles Consider a base class and a derived class:

```
class parent;
endclass

class child extends parent;
endclass
```

A parent handle can refer to a child object:

```
parent p;
child c;

c = new();
p = c;
```

The object is actually a `child`, even though it is accessed through the `parent` handle.
We can safely recover the child handle using:

```
child c_cast;

if ($cast(c_cast, p)) begin
    // Cast succeeded
end
```

Successful Cast When the object referenced by the source handle is actually a `child`:

```
c = new();
p = c;

$cast(c_cast, p);
```

the cast succeeds because the actual object is compatible with `child`. The child-specific methods can then be accessed through `c_cast`.

Failed Cast If the source handle refers to a real `parent` object:

```
p = new();
```

then:

```
$cast(c_cast, p)
```

fails because the actual object is not a `child`. The return value allows the testbench to detect this safely:

```
if ($cast(c_cast, p))
    $display("SUCCESS");
else
    $display("FAILED");
```

Static Cast vs `$cast`

Feature	Static Cast	`$cast`
Syntax	`type'(value)`	`$cast(dest, source)`
Evaluation	Static/explicit conversion	Runtime check
Return value	Cast value	`1` or `0`
Common use	Type conversion	Class-handle casting
Failed cast detection	Not through a success return	Yes

Why `$cast` Matters in Verification `$cast` is important in object-oriented verification because testbenches frequently use:

- Base-class handles
- Derived transaction classes
- Polymorphism
- Factory-created objects
- Different transaction types handled through common interfaces

The main idea is:

> **`$cast` checks at runtime whether the source object can safely be treated as the destination type.**

1.17.2 Complete Runnable Code

Code type: SystemVerilog simulation/demo — class-based testbench example, no DUT.

```
module sv_011_cast;

    class parent;
        virtual function void show();
            $display("Parent object");
```

```
        endfunction
    endclass

    class child extends parent;
        function void show();
            $display("Child object");
        endfunction

        function void child_method();
            $display("Child-specific method");
        endfunction
    endclass

    parent p;
    child  c;
    child  c_cast;

    initial begin
        // Create a child object through a parent handle
        c = new();
        p = c;

        // Safe dynamic cast: succeeds
        if ($cast(c_cast, p)) begin
            $display("Cast 1: SUCCESS");
            c_cast.child_method();
        end
        else begin
            $display("Cast 1: FAILED");
        end

        // Create a real parent object
        p = new();

        // Safe dynamic cast: fails
        if ($cast(c_cast, p)) begin
            $display("Cast 2: SUCCESS");
        end
        else begin
            $display("Cast 2: FAILED");
        end
    end

endmodule
```

1.17.3 Expected Output

```
Cast 1: SUCCESS
Child-specific method
Cast 2: FAILED
```

1.17.4 Key Syntax

```
$cast(destination, source)
```

Typical usage:

```
if ($cast(child_handle, parent_handle)) begin
    // Cast succeeded
end
else begin
    // Cast failed
end
```

Remember:

- `type'(expression)` → **static casting**
- `$cast(destination, source)` → **dynamic/safe casting**
- `$cast` is particularly important for **class handles and polymorphism**

1.18 SV-012 — System Functions

SystemVerilog provides several built-in system functions for determining the **size, width, and boundaries** of data objects and arrays.

The important functions in this topic are:

- `$bits`
- `$size`
- `$clog2`
- `$left`
- `$right`
- `$high`
- `$low`

These functions are particularly useful in verification because they reduce hard-coded widths and array boundaries.

1.18.1 `$bits`

`$bits` returns the number of bits required to represent a data type or expression.

```
$bits(data)
```

For example:

```
logic [7:0] data;
```

has 8 bits:

```
$bits(data) = 8
```

For:

```
logic [15:4] range_data;
```

there are 12 bits:

```
15 - 4 + 1 = 12
```

So:

```
$bits(range_data) = 12
```

1.18.2 `$size`

`$size` returns the number of elements in an array.

For:

```
int array [0:4];
```

there are five elements:

```
0, 1, 2, 3, 4
```

Therefore:

```
$size(array) = 5
```

1.18.3 $clog2

`$clog2` returns the **ceiling of log base 2** of its argument.

```
$clog2(10)
```

Since:

```
2^3 = 8
2^4 = 16
```

the smallest power of two that is at least 10 is 16.

Therefore:

```
$clog2(10) = 4
```

It is commonly used to calculate address widths and counter widths.

For example:

```
localparam int ADDR_WIDTH = $clog2(DEPTH);
```

1.18.4 $left and $right

These functions return the leftmost and rightmost index of an array or packed/unpacked dimension.

For:

```
int array [0:4];
```

the results are:

```
$left(array)  = 0
$right(array) = 4
```

The result depends on the declared direction of the dimension.

For example, a descending range would behave differently:

```
logic [7:0] data;
```

Here:

```
$left(data)  = 7
$right(data) = 0
```

1.18.5 $high and $low

`$high` returns the highest index of a dimension. `$low` returns the lowest index of a dimension.

For:

```
int array [0:4];
```

the results are:

```
$high(array) = 4
$low(array)  = 0
```

Unlike `$left` and `$right`, `$high` and `$low` describe the numerical highest and lowest index. For example:

```
logic [7:0] data;
```

has:

```
$left(data)  = 7
$right(data) = 0
$high(data)  = 7
$low(data)   = 0
```

1.18.6 `$left` / `$right` vs `$high` / `$low`

Function	Meaning
`$left`	Leftmost declared index
`$right`	Rightmost declared index
`$high`	Numerically highest index
`$low`	Numerically lowest index

This distinction becomes important when working with both ascending and descending ranges.

1.18.7 Why These Functions Matter in Verification

These system functions are useful for:

- Automatically determining signal widths
- Calculating address widths
- Handling arrays without hard-coded sizes
- Writing reusable testbench code
- Iterating through different array ranges
- Making verification code parameterized and scalable

1.18.8 Complete Runnable Code

Code type: SystemVerilog simulation/demo — no DUT.

```
module sv_012_system_functions;

    logic [7:0] data;
    logic [15:4] range_data;
    int array [0:4];

    initial begin
        // $bits: number of bits in a data type/object
        $display("$bits(data)       = %0d", $bits(data));
        $display("$bits(range_data) = %0d", $bits(range_data));

```

```
12          // $size: number of elements in an array
13          $display("$size(array)      = %0d", $size(array));
14
15          // $clog2: ceiling of log2
16          $display("$clog2(10)        = %0d", $clog2(10));
17
18          // Array/range boundary functions
19          $display("$left(array)      = %0d", $left(array));
20          $display("$right(array)     = %0d", $right(array));
21          $display("$high(array)      = %0d", $high(array));
22          $display("$low(array)       = %0d", $low(array));
23      end
24
25  endmodule
```

1.18.9 Expected Output

```
$bits(data)       = 8
$bits(range_data) = 12
$size(array)      = 5
$clog2(10)        = 4
$left(array)      = 0
$right(array)     = 4
$high(array)      = 4
$low(array)       = 0
```

1.18.10 Key Syntax

```
$bits(expression)
$size(array)
$clog2(value)
$left(array)
$right(array)
$high(array)
$low(array)
```

The main idea is to use these functions to **query dimensions and calculate sizes dynamically instead of hard-coding them**.